CLAFS — Campus Lost-and-Found System

API Documentation / Notes

Phase 3 submission · ITS122P – AM2 · Group 4

Samuela Ysebelle Adame · Kervin Del Rosario · Mariah Kate Guiang · Kendrick Sebastian

Mapua University

Contents

1. Overview 2
2. CLAFS JSON API (internal) 2
2.1 API purpose 2
2.2 API endpoints 2
2.3 Data retrieved from the API 6
2.4 How the API is integrated into the website 6
3. Nager.Date Public Holiday API (external) 7
3.1 API used 7
3.2 API purpose 7
3.3 API endpoint 7
3.4 Data retrieved 7
3.5 How it is integrated 8
4. Where to see the API working 8

1. Overview

CLAFS uses two APIs. The first is the CLAFS JSON API — a set of PHP endpoints in the api/ folder that the website's own JavaScript calls with fetch() to read and write lost-and-found records without reloading the page. The second is an external, third-party API — Nager.Date public holidays — which the browser calls directly to show when the Lost & Found office is closed.

API	Type	Base URL	Format	Auth
CLAFS JSON API	Internal (own back end)	`/api/` e.g. http://localhost:8000/api/	JSON	PHP session cookie (log in first)
Nager.Date Public Holiday API	External / third-party	`https://date.nager.at/api/v3/`	JSON	None (free, no key)

Files involved: `public/js/api.js` (fetch() wrappers for every call), `public/js/app.js` (form handlers, live search, holiday widget), `api/*.php` (server endpoints), `config/db_connect.php` (shared validation, JSON helpers, database access).

2. CLAFS JSON API (internal)

2.1 API purpose

The CLAFS JSON API is the bridge between the browser and the MySQL database. Every action that changes data — filing a lost report, logging a found item, submitting or reviewing a claim, changing a status, managing users — is sent to one of these endpoints with fetch() and the response is used to update the page in place. The public found-items search also reads from it, so results appear as the user types.

Rules that protect the data (who may see or change what) are enforced on the server in each endpoint; the JavaScript validation in app.js is only a convenience for the user.

2.2 API endpoints

Endpoint	Method	Who	Purpose
`api/get_items.php`	GET	Anyone (found) / logged in (lost)	List found items or lost reports with keyword, category, status, date and sort filters
`api/add_item.php`	POST (multipart)	Logged in (lost) / staff (found)	Create a lost report or found item, with an optional photo upload. Lost reports are rate-limited per account and checked for duplicates
`api/update_item.php`	POST (multipart)	Owner (lost) / staff (found)	Edit an existing report or item; optionally replace the photo
`api/update_status.php`	POST	Staff; owner may close own report	Change a status (stored/returned/disposed, open/matched/closed). "Returned" also

Endpoint	Method	Who	Purpose
			closes the linked report and rejects other pending claims
`api/claims.php`	POST	Logged in / staff	action=create (submit a claim), review (approve/reject), withdraw
`api/moderate_report.php`	POST	Staff	Reject a lost report, mark it as spam or as a confirmed false report (warning, then deactivation), or remove a false/spam report (soft delete)
`api/update_user.php`	POST	Admin	Change a user's role, unlock the account, or deactivate/reactivate it (reactivating after repeated false reports needs review notes)

Common response shape. Every endpoint returns JSON with an `ok` flag. Errors use the matching HTTP status code (401 not logged in, 403 not allowed, 404 not found, 409 conflict, 422 validation) and a message; validation errors also include an `errors` object keyed by field name so the form can highlight the exact field.

```
// success                          // validation failure (HTTP 422)
{ "ok": true, ... }                  { "ok": false,
                                       "error": "Please fix the highlighted fields.",
                                       "errors": { "description": "Must be at least 20
characters." } }
```

GET /api/get_items.php

Query parameters: `type`=found|lost, `q` (keyword), `category`, `status`, `from`, `to` (dates), `sort`=newest|oldest, `limit` (1–100).

```
GET /api/get_items.php?type=found&q=key&sort=newest&limit=100

{ "ok": true, "type": "found", "count": 1,
  "items": [ { "item_id": 5, "item_name": "Silver keychain with 3 keys", "category": "Keys",
               "description": "Keychain with three keys found near the motorcycle parking
area.",
               "location_found": "Parking Area", "date_found": "2026-09-11",
               "image_url": null, "status": "stored", "created_at": "2026-09-11 08:50:00" }
] }
```

POST /api/add_item.php and /api/update_item.php

Sent as `multipart/form-data` (a `FormData` built from the form) so the photo file can be included. Fields: `type`, `item_name`, `category`, `description`, `date_lost` + `location_lost` (lost) or `date_found` + `location_found` + `storage_location` + `private_details` (found), optional `photo`. update_item.php also takes `id`.

`category` must be one of `CATEGORIES` and `location_lost` / `location_found` one of the campus locations (`CAMPUS_LOCATIONS` in `config/config.php`): Main Entrance & Lobby, Hallways & Elevators, Classrooms, Computer Labs, Science & Health Labs, Media Studios, Library, Student Lounge, Canteen & Café, Gymnasium, Cinema, Prayer Room, Student Services Office, Restrooms, Parking Area, Other. Any other value is refused with HTTP 422 and `"Choose a valid category."` or `"Choose a location from the list."`.

Choosing Other requires a description in `category_other`, `location_lost_other` or `location_found_other` (up to 50 characters for a category, 150 for a location). That text is stored in place of "Other"; if it is missing, HTTP 422 with `"Describe what "Other" is."` on the `_other` field. The form keeps the text box disabled unless Other is selected. Category filters and the statistics count these typed values as Other.

`storage_location` (staff intake and edit) must be one of `STORAGE_LOCATIONS`: Cabinet A, Shelf 1; Cabinet B, Shelf 2; Cabinet C, Shelf 1; Drawer 1 (IDs); Drawer 2 (Keys); Bin 4 (Misc). Anything else gets HTTP 422 and `"Choose a storage location from the list."`.

`date_lost` / `date_found` must be between one year ago and today: a later date gets `"Date cannot be in the future."`, an earlier one `"Date cannot be more than a year ago."` (HTTP 422).

```
POST /api/add_item.php        (type=lost, item_name=..., photo=<file>)

HTTP 201   { "ok": true, "type": "lost", "id": 6, "url": "/view_item.php?type=lost&id=6",
"item": { ... } }
```

Before a lost report is saved the server locks the account's row, so parallel or double-clicked requests run one at a time, and then checks, in order: the submission limit (`REPORT_LIMIT_MAX` reports per `REPORT_LIMIT_WINDOW_MINUTES`, 3 per 60 minutes) → HTTP 429 with `retry_after` in seconds; an exact copy of one of the user's own open reports → HTTP 409 with `existing`; a similar report (same category, similar name or description, same place or a date within a week) → HTTP 409 with `duplicate: true` and the matching reports. Sending `confirm_duplicate=1` files a similar report anyway. Other users' reports are never compared.

```
HTTP 429  { "ok": false, "retry_after": 2700,
            "error": "You have reached the report submission limit (3 reports
                      per 60 minutes). You can submit another report in
                      approximately 45 minutes." }

HTTP 409  { "ok": false, "duplicate": true,
            "error": "You may already have a similar report. ...",
            "matches": [ { "report_id": 9, "item_name": "Black Wallet",
                           "location_lost": "Library", "date_lost": "2026-10-02",
                           "status": "open", "url": "/view_item.php?type=lost&id=9" } ] }
```

POST /api/update_status.php

```
POST /api/update_status.php    Content-Type: application/json
```

```
{ "type": "lost", "id": 9, "status": "closed" }

{ "ok": true, "type": "lost", "id": 9, "status": "closed",
  "message": "Report #9 is now “Closed”." }
```

For lost reports, `status=matched` also accepts `item_id` — the found item staff matched the report to — which is stored in `lost_reports.matched_item_id` and shown to the owner.

Staff review a student's found-item post with `{ "type": "found", "id": 7, "moderation": "approved" | "rejected", "storage_location": "Drawer 1 (IDs)", "review_note": "..." }`. Approving needs a `storage_location` from `STORAGE_LOCATIONS`; rejecting needs a `review_note` of at least 10 characters. Otherwise HTTP 422 with the field in errors. On the item page the Approve and Reject buttons stay disabled until their field is filled in.

POST /api/claims.php

```
{ "action": "create", "item_id": 4, "proof_description": "...", "report_id": 1,
"confirm_truth": 1 }
{ "action": "review", "claim_id": 4, "decision": "approve", "review_note": "Bring a valid
ID ..." }
{ "action": "withdraw", "claim_id": 5 }

{ "ok": true, "action": "review", "claim_id": 4, "status": "approved",
  "message": "Claim #4 approved. The claimant can now see your note.", "claim": { ... } }
```

POST /api/update_user.php

```
{ "user_id": 4, "role": "staff" }        or        { "user_id": 4, "is_active": 0 }

{ "ok": true, "user_id": 4, "role": "staff", "is_active": 1, "message": "Bea Lim: role set
to Security & Maintenance." }
```

Reactivating an account that was deactivated for repeated false reports also needs `note` (10–1000 characters) recording the in-person review at the Lost & Found office; without it the endpoint answers HTTP 422. The note is saved in the activity log with the admin’s name. Account holders have no self-service way back.

POST /api/moderate_report.php

Staff only. `action` is `rejected` (no penalty), `false_report`, `spam` (no penalty by itself) for an open or matched report, or `delete` for a report already marked false or spam. Every action needs a `reason` of 10–1000 characters. A false report is recorded in `account_violations`: the first one warns the reporter (email and dashboard banner), the second deactivates the account and ends its sessions. Moderating a report twice answers HTTP 409; students, faculty and admins get HTTP 403.

```
{ "report_id": 7, "action": "false_report",
  "reason": "Same prank wallet report filed again with different wording." }

{ "ok": true, "report_id": 7, "status": "false_report",
  "message": "Report #7 marked as a false report. This was the reporter's violation
```

```
number 2, so their account has been deactivated and logged out." }
```

2.3 Data retrieved from the API

Endpoint	Data returned	Used for
get_items.php	Array of found items (item_id, item_name, category, description, location_found, date_found, image_url, status) or lost reports. Staff also receive storage_location and private_details.	Rendering the item cards on the Found Items page as the user searches
add_item.php / update_item.php	The saved record plus its id and page URL; or the rate-limit message, or the similar reports	Redirecting to the new/updated item page; listing similar reports with View existing reports / Continue anyway / Cancel
update_status.php	Record type and id, its new status and a message	Updating every status badge on the page and showing a banner
claims.php	The claim row, its new status and a message	Replacing the form with a confirmation, revealing the hand-over step, removing a withdrawn claim
update_user.php	Updated role / active flag and a message	Updating the Active/Deactivated badge and flipping the button
moderate_report.php	Report id, its new status (rejected, false_report, spam or deleted) and a message saying whether the reporter was warned or deactivated	Updating the report's status badge and replacing the moderation form with a confirmation

2.4 How the API is integrated into the website

Client side. `public/js/api.js` defines a global `ClafsApi` object with one function per endpoint (`getItems`, `addItem`, `updateItem`, `updateStatus`, `createClaim`, `reviewClaim`, `withdrawClaim`, `moderateReport`, `updateUser`). Each one calls `fetch()`, sends JSON or FormData, parses the JSON reply, and rejects with an Error that carries the HTTP status and the field errors when the server says `ok:false`.

```
// public/js/api.js (excerpt)
return fetch(url, init).then(function (response) {
    return response.json().then(function (data) {
        if (!response.ok || data.ok === false) {
            var error = new Error(data.error); error.status = response.status; error.data =
data;
            throw error;
        }
        return data;
    });
});
```

Forms. A form opts in with `data-api="add_item"` (or update_item, update_status, claim, review, withdraw, moderate_report, update_user). `app.js` intercepts its submit event, runs client-side validation,

calls the matching `ClafsApi` function, and then — instead of reloading — updates the page: badges with `data-status-for="found-2"` change text and colour, `data-done="replace"` swaps the form for a success alert, `data-done="remove"` deletes the card. Server-side validation errors (HTTP 422) are written under the exact field they belong to.

The report form also has `data-submit-guard`: its submit button stays disabled until every field passes the same checks, and a duplicate warning (HTTP 409 with `duplicate`) is shown in the form with View existing reports / Continue anyway / Cancel. These are conveniences; the endpoint repeats every check.

Live search. The search form on `browse.php` has `data-live-search`. Typing in the keyword box (debounced 300 ms) or changing a filter fetches `browse.php` with the same filters and swaps in its result cards, so the cards come from the same PHP template as a normal page load; the URL is updated with `history.replaceState` so the search can still be shared. Without JavaScript the same form submits normally and PHP renders the results.

Server side. Each `api/*.php` file requires `config/db_connect.php`, checks the session and role, validates input with the shared `validate_item_input()` helper, runs prepared PDO statements against MySQL, and replies with `json_response()` / `json_error()`. The endpoints accept ordinary form fields as well as JSON, so they also work without JavaScript.

3. Nager.Date Public Holiday API (external)

3.1 API used

Nager.Date — a free, open public-holiday web service (`https://date.nager.at`). No registration or API key is required and the API allows cross-origin requests, so the browser can call it directly.

3.2 API purpose

The Lost & Found office is closed on Philippine public holidays. The website fetches the official holiday list and shows the next closure date on the landing page (Office hours card) and in the Pickup details of every found item, so students know when they can actually collect an item.

3.3 API endpoint

```
GET https://date.nager.at/api/v3/PublicHolidays/{year}/PH

e.g. GET https://date.nager.at/api/v3/PublicHolidays/2026/PH
```

3.4 Data retrieved

A JSON array with one object per holiday, sorted by date. Fields used by CLAFS are date, name and localName.

```
[ { "date": "2026-10-31", "localName": "All Saints' Day Eve", "name": "All Saints' Day Eve",
```

```
    "countryCode": "PH", "global": true, "types": ["Public"] },
  { "date": "2026-11-01", "localName": "Undas", "name": "All Saints' Day", ... }, ... ]
```

3.5 How it is integrated

Elements marked `data-next-holiday` (index.php and view_item.php) are placeholders. On page load `app.js` calls `ClafsApi.getHolidays(year)`, which fetches the endpoint above with `fetch()`. The script keeps the holidays on or after today (falling back to next year's list when none are left), takes the first one, and writes "Next closure: Oct 31, 2026 — All Saints' Day Eve" into the placeholder. If the request fails (offline, service down) the placeholder shows "Holiday schedule unavailable right now." and the rest of the page is unaffected.

```
// public/js/app.js (excerpt)
ClafsApi.getHolidays(year)
    .then(function (holidays) {
        var upcoming = holidays.filter(function (h) { return h.date >= todayIso; });
        return upcoming.length ? upcoming : ClafsApi.getHolidays(year + 1);
    })
    .then(function (upcoming) { node.innerHTML = "Next closure: <strong>" +
formatDate(upcoming[0].date) + "</strong> — " + upcoming[0].localName; })
    .catch(function () { node.textContent = "Holiday schedule unavailable right now."; });
```

4. Where to see the API working

Page	What to try	API call made
Found Items (browse.php)	Type in the search box or change a filter	GET browse.php (same page, same filters) – cards update without reload
Home page (index.php)	Look at the Office hours card	GET date.nager.at … /PH — next closure appears
Report lost item (report.php)	Submit the form (try leaving fields empty, then the same report twice, then a fourth report within an hour)	POST add_item.php — field errors, duplicate warning or limit message / redirect to the new report
Item page (view_item.php)	Submit a claim as a student; approve it and mark returned as staff	POST claims.php, POST update_status.php — badges update in place
Lost report page (view_item.php?type=lost, staff)	Reject a report, or mark it as spam or false; then remove it	POST moderate_report.php — badge updates, form replaced by a confirmation
Manage found items (browse.php?manage=1)	Change a status in the table	POST update_status.php
Users (index.php?tab=users, admin)	Change a role or deactivate an account	POST update_user.php
Account review (index.php?tab=users&id=N, admin)	Reactivate an account deactivated for false reports, with review notes	POST update_user.php with note

Open the browser's developer tools (F12 → Network tab) while doing any of the above to watch the fetch() requests and JSON responses.